Arctos Robotic Arm — Motor and CAN Reference Guide

Consolidated from project debugging logs, source-code inspection,
and empirical bench testing

September 2026

Contents

1 About This Guide **2**

2 Core Concepts **2**
- 2.1 CAN bus, in one paragraph . . . 2
- 2.2 Gearboxes and closed-loop steppers, in one paragraph . . . 2

3 Hardware **2**
- 3.1 Joint, motor and CAN ID map . . . 2
- 3.2 Motor specifications . . . 3
- 3.3 CANable 2 adapter . . . 3
- 3.4 Bus termination check (power off) . . . 4

4 The CAN Interface on Ubuntu **4**

5 Controller Panel Configuration **5**

6 The MKS CAN Protocol **6**
- 6.1 Frame structure and checksum . . . 6
- 6.2 Command reference . . . 6
- 6.3 Reading the encoder . . . 7
- 6.4 Encoder non-linearity . . . 7
- 6.5 Motion commands . . . 7
- 6.6 Speed and its limits . . . 8
- 6.7 Measured ceilings, per joint . . . 9

7 Converting Encoder Counts to Joint Degrees **9**

8 Safety **10**

9 Troubleshooting **10**
- 9.1 Total silence with a powered driver: the RS485 variant . . . 11

10 What This Guide Supersedes **11**

1 About This Guide

This is the *reference* document for the Arctos arm's motors and CAN bus: how the bus works, how the protocol is framed, how to convert encoder counts to joint angles, and what has actually been verified against hardware.

It is deliberately **not** a step-by-step procedure. Two companion SOPs cover that, and each one points back here for background:

- `sop_nema17.tex` — bringing up a NEMA 17 joint (MKS Servo42D). Joints A, B and C.
- `sop_nema23.tex` — bringing up a NEMA 23 joint (MKS_57D). Joint X.

This guide supersedes `arctos_joint_b_guide.tex`, which was removed. All of its generally-applicable content was folded in here, with several items **corrected** in the process — see Section 10 for exactly what changed and why, so that nothing is silently re-derived from the old version.

2 Core Concepts

2.1 CAN bus, in one paragraph

CAN is a differential two-wire multi-drop bus. Every device shares the same pair (`CAN_H` and `CAN_L`) plus a common ground, and each device is addressed by an ID. Every node on the bus must agree on the bitrate; the Arctos arm runs at **500 kbit/s**. Because all nodes share one pair, two devices configured with the same CAN ID will collide and neither will behave correctly.

2.2 Gearboxes and closed-loop steppers, in one paragraph

Each joint is a stepper motor driving a reduction gearbox. The reduction multiplies torque and divides speed and angular error: with a 67.82:1 gearbox, one motor revolution is only $360/67.82 \approx 5.31°$ at the joint. The MKS controllers are *closed-loop*: a magnetic encoder on the motor shaft reports true position, so the controller can detect that it has fallen behind. Two consequences matter constantly in practice — the encoder measures the **motor shaft, not the joint output**, and a stalled stepper converts its full phase current into heat with no motion to carry it away.

3 Hardware

3.1 Joint, motor and CAN ID map

Caution: Always confirm a joint's CAN ID against real hardware

This project's documents have carried wrong CAN IDs for Joint A and Joint C at various times, and Joint C's motor was found at `0x05` when both the config and the docs said otherwise. Run `can_id_scan.py` — which sends only the read-only `0x31` query and never enables coils — before trusting any table, including this one.

Joint	Role	CAN ID	Motor / driver	Gear ratio
X	Base/waist rotation	`0x01`	NEMA 23 / MKS_57D	13.5:1
Y	(not confirmed)	`0x02`	—	150.0:1
Z	(not confirmed)	`0x03`	—	150.0:1
A	Elbow	`0x04`	NEMA 17 / Servo42D	48.0:1
B	Wrist rotation	`0x05`	NEMA 17 / Servo42D	67.82:1
C	Wrist joint	`0x06`	NEMA 17 / Servo42D	67.82:1

Table 1: Joint map. CAN IDs and gear ratios match `arctos_controller.yaml`. Joint A's `0x04` and its 48:1 ratio are empirically verified; the others are config values and should be confirmed with `can_id_scan.py` before use.

Parameter	NEMA 17 (Servo42D)	NEMA 23 (MKS_57D)
Holding torque	24 N·cm	1.8 N·m
Rated current	1.2 A	2.8 A
Shaft diameter	5 mm	6.35 mm
Step angle	1.8° (200 full steps/rev)	1.8° (200 full steps/rev)
Microstepping	16× (3200 pulses/rev)	16× (3200 pulses/rev)
Encoder	16,384 counts/rev (14-bit)	16,384 counts/rev (14-bit)
Joints	A, B, C	X

3.2 Motor specifications

Caution: Do not confuse motor steps, microstep pulses, and encoder counts

Three different resolutions are in play and mixing them is the single most common arithmetic error in this project:

- **200** full steps per motor revolution — the physical stepping resolution.
- **3200** microstep pulses per motor revolution — the scale that `POSITION_CONTROL (0xFD)` commands are expressed in.
- **16,384** encoder counts per motor revolution — the scale that `READ_ENCODER (0x31)` reports in.

One microstep pulse is 16384/3200 = 5.12 encoder counts. Commanded motion and measured motion are therefore *never* in the same units.

3.3 CANable 2 adapter

- USB VID:PID `16d0:117e`.
- Contains its own CAN controller — **do not** add an external MCP2515 module in series with it.
- Only three wires are needed between adapter and controller: `CAN_H`, `CAN_L`, `GND`. Do *not* connect the CANable's power lines to the motor controller.

```
Ubuntu host
     |
    USB
     |
 CANable 2
     |
CAN_H / CAN_L / GND
     |
MKS controller (Servo42D or 57D)
     |
  Joint motor
```

Listing 1: Physical signal path

3.4 Bus termination check (power off)

With the system powered off, measure resistance between `CAN_H` and `CAN_L`:

Reading	Meaning
$\approx 60\,\Omega$	Correct — two 120Ω terminators in parallel
$\approx 120\,\Omega$	Only one terminator present
Infinite / open	Wiring is disconnected
Very low	Possible short

4 The CAN Interface on Ubuntu

The CANable presents as a serial device (`slcan`), which is bridged to a Linux network interface by `slcand`. This must be rebuilt every time the adapter is plugged in or re-enumerates.

```
lsusb                                        # expect: ID 16d0:117e CANable2
ls -l /dev/serial/by-id/                     # stable name, survives renumbering
sudo pkill slcand
sudo ip link delete can0 2>/dev/null
sudo slcand -o -c -s6 /dev/serial/by-id/<your-canable> can0
sudo ip link set can0 txqueuelen 1000
sudo ip link set up can0
ip -br link show can0                        # expect: can0  UP
```

Listing 2: Bring-up

Caution: The `slcand` bitrate flag is a coded index, not a number

`-s0`=10k, `-s1`=20k, `-s2`=50k, `-s3`=100k, `-s4`=125k, `-s5`=250k, `-s6`=**500k**, `-s7`=800k, `-s8`=1M. The Arctos bus is 500 kbit/s, so `-s6` is correct. Earlier project scripts used `-s3`, which silently attached at 100 kbit/s.

Caution: On slcan, the bitrate is set at attach time and `ip` cannot show it

Do *not* use `ip link set can0 up type can bitrate 500000`. That netlink attribute is for native CAN controller drivers and is not supported on slcan interfaces. Consequently `ip -details`

`link show can0` reports `bitrate 0` and `clock 0` on a perfectly healthy CANable — this is normal and is **not** a fault to chase. An older version of this guide told the reader to verify by looking for "`bitrate 500000`"; that check can never pass on this hardware.

Caution: Kernel CAN error counters are meaningless on slcan

The `re-started / bus-errors / arbit-lost / error-warn / error-pass / bus-off` counters in `ip -details -statistics link show can0` are not populated by the slcan driver — they read zero regardless of what the bus is doing. Do not use them as evidence that a bus is healthy, and note that the "total silence with zero bus errors implies no CAN transceiver" heuristic (Section 9.1) requires a native CAN controller to be meaningful.

```
# 1. Stop any running control script (Ctrl+C)
sudo ip link set can0 down
sudo pkill slcand
# then physically unplug the adapter
```

Listing 3: Shutdown

5 Controller Panel Configuration

Confirm these on the controller's own display before attempting CAN communication:

Setting	Value
CAN ID	the joint's ID from the map above
CAN Rate	`500` (kbit/s)
CAN Response (CANRSP)	`ENABLE`
CAN CRC	`SUM-8`
Working current	at or below the motor's rated current
Working mode	see caution below

Save and power-cycle the controller after any change. The CAN ID in particular must be committed, not merely displayed.

Caution: Set the working current to the motor's rating, not the panel default

A controller was found in this project storing a 3000 mA default against a motor rated 1.2 A — roughly 2.5×. Check this on every new controller.

Caution: Working-mode mismatch can imitate a communication fault

The ROS 2 driver defines six modes and defaults to `CR_vFOC` (2):

```
enum class MotorMode {
    CR_OPEN = 0, CR_CLOSE = 1, CR_vFOC = 2,
    SR_OPEN = 3, SR_CLOSE = 4, SR_vFOC = 5
```

```
};
```

If the panel's mode disagrees with what a motion command assumes, the controller can accept and checksum-acknowledge a frame without rotating the shaft. Confirm the mode before concluding a "no motion" result is a wiring or CAN problem.

6 The MKS CAN Protocol

6.1 Frame structure and checksum

Every frame is `[command byte] [data bytes...] [checksum byte]`, where the checksum is an 8-bit sum that **includes the CAN ID**:

```python
def checksum(motor_id, data_bytes):
    return (motor_id + sum(data_bytes)) & 0xFF
```

Frame (no checksum)	CAN ID	Arithmetic	Checksum
31	0x04	0x31+0x04	35
31	0x05	0x31+0x05	36
3A	0x05	0x3A+0x05	3F
F3 01	0x05	0xF3+0x01+0x05	F9
F7	0x06	0xF7+0x06	FD

Table 2: Worked checksum examples. Note the checksum differs per joint — a frame copied from another joint's documentation will be silently rejected.

Caution: Never hand-invent a checksum, and never reuse another joint's frame

Omitted or incorrect checksum bytes were the direct cause of several "the motor ignored my command" failures in this project. Compute it from the formula every time, including for the emergency stop.

6.2 Command reference

Name	Byte	Status
QUERY_MOTOR	0xF1	Liveness check. Response `F1 01 <crc>`.
READ_ENCODER	0x31	**Verified.** Use this one. See below.
READ_ENCODER (carry form)	0x30	Superseded — see Section 10.
READ_VELOCITY	0x32	Responds; zero at rest. Layout unconfirmed.
READ_ERROR	0x39	Signed 32-bit position deficit. Grows when stalled.
READ_ENABLE_STATE	0x3A	01=enabled, 00=disabled. Verified.
READ_SHAFT_PROTECTION_STATE	0x3E	00=clear. Non-zero latches the driver.
SET_WORKING_MODE	0x82	See panel caution.
ENABLE_SHAFT_PROTECTION	0x88	**Avoid** — false positives, latches.

Name	Byte	Status
ENABLE_MOTOR	0xF3	01=enable, 00=disable. Verified.
RELATIVE_POSITION	0xF4	Unsafe packing — see motion section.
ABSOLUTE_POSITION	0xF5	**Unsafe** — see motion section.
SPEED_CONTROL	0xF6	Not exercised.
EMERGENCY_STOP	0xF7	Verified with checksum.
POSITION_CONTROL	0xFD	**Verified. Preferred for all motion.**

6.3 Reading the encoder

Use `0x31`. The response is `[0x31][6 bytes of signed position][checksum]`, where the six bytes are a big-endian signed 48-bit multi-turn count on the 16,384-counts-per-revolution scale:

```
raw48 = int.from_bytes(data[1:7], byteorder="big", signed=True)
motor_degrees = raw48 * 360.0 / 16384
joint_degrees = motor_degrees / gear_ratio
```

Listing 4: Verified position extraction

This is multi-turn and does not wrap at one revolution. It is reset to approximately zero when the controller is power-cycled, so it is a *relative* reference from power-on, not an absolute machine coordinate.

Caution: The encoder measures the motor shaft, not the joint output

Gearbox backlash and any lost motion downstream of the motor are invisible to this encoder. It cannot be used to measure output-side backlash; that needs a dial indicator on the output.

6.4 Encoder non-linearity

The magnetic encoder carries a once-per-revolution sinusoidal error caused by the sensing magnet sitting slightly off-centre on the shaft. Measured on Joint A over 6,725 samples: **1.465° peak-to-peak at the motor shaft**, about 0.407% of a revolution.

Two practical consequences:

- It cancels over any whole number of motor revolutions. Choosing a step size that is an integer number of motor turns makes it vanish — Joint A steps of exactly 6 motor revolutions each landed at 100.0% every time, where 90° motor-shaft steps scattered 98.7–101.4%.
- Through a reduction it becomes negligible at the joint: 1.465° through 48:1 is 0.031°.

Repeatability is unaffected — returning to the same shaft angle repeats to better than one encoder count. Absolute readings carry the cyclic error; relative and repeat positioning do not.

6.5 Motion commands

Use `POSITION_CONTROL (0xFD)`. It is the command the project's own working jog code uses, and the only one verified safe here.

```
[0xFD]
[direction | ((speed >> 8) & 0x0F)]   direction: 0x00 forward, 0x80 reverse
[speed & 0xFF]                        speed is approximately rpm (see below)
[accel]
[(pulses >> 16) & 0xFF]               ALWAYS-POSITIVE magnitude,
[(pulses >>  8) & 0xFF]               on the 3200-pulses-per-motor-rev scale
[pulses & 0xFF]
[checksum]
```

Listing 5: 0xFD frame layout

$$\text{pulses} = |\text{joint degrees}| \times \frac{3200 \times \text{gear\_ratio}}{360}$$

The driver replies with an unsolicited two-frame status sequence: `FD 01` ("started") and then, if the commanded distance was actually achieved, `FD 02` ("complete"). **Absence of `FD 02` is the reliable signal that a command did not finish.**

Caution: Do not use `0xF5`, and be careful with `0xF4`

`0xF5` packs an absolute target through a plain `& 0xFFFFFF`. On a joint whose raw counter had drifted to roughly −274,000 counts, a small intended nudge became an enormous, non-settling, uninterruptible move that required cutting motor power. `0xF4` packs a possibly-negative delta the same way. The `0xFD` pattern above — a separate direction byte plus an always-positive magnitude — does not have this failure mode.

Caution: Never retry a stalled command into an obstruction

Re-sending the same command to a joint that has not moved does not change the situation; it only dumps stall current into the motor as heat. Nine consecutive full-torque retries into a hard stop drove one joint's internal position error to −124,374 counts. If a step under-delivers, stop, or back off and re-attempt — do not repeat blindly.

6.6 Speed and its limits

The `speed` field is very close to motor rpm. Measured on a NEMA 17 with no load:

Commanded	Actual	Behaviour
10–225	tracks to within 0.5%	linear; `rpm` = 1.001× `speed`
250	98.7%	edge of linear
275–325	96.5%–90%	rolling off
350+	saturates ≈310 rpm	losing steps

There is no hard stall — speed rolls off gracefully and plateaus. **But above roughly 250 the motor receives more pulses than it executes, so position is silently lost.** For anything where position matters, stay at or below 225.

6.7 Measured ceilings, per joint

The table above is a bare motor on a bench. A joint drags its gearbox with it and the ceiling falls accordingly, so treat the no-load numbers as an upper bound rather than a specification, and measure each joint with `joint_speed_limit_test.py` instead of inheriting them.

Joint	Driver	Gear	Linear to	Saturates
— (bench, no load)	Servo42D	none	225 rpm	≈310 rpm
B	Servo42D	67.82:1	150 rpm	≈165 rpm

Table 4: Joint B measured 2026-09-02 over a 48° arc at `accel=8`. Commanded 150 tracked at 99.7%; commanded 165, 175, 200, 225 and 300 all produced 159–165 rpm. Past saturation, extra commanded speed buys nothing but current.

Caution: Over-commanding speed did not lose position on a geared joint

`0xFD` commands a pulse *count*, not a duration. On Joint B every trial from 150 through 300 commanded delivered its 48° arc to +0.01%: past saturation the move simply takes longer, it does not drop pulses. Silent position loss requires actual slip. So do not carry the no-load "loses steps above 250" result across to a geared joint automatically — verify which behaviour the joint in front of you actually shows before relying on either.

7 Converting Encoder Counts to Joint Degrees

$$\text{counts per joint degree} = \frac{\text{gear\_ratio} \times 16384}{360}$$

Gear ratio	Counts per joint degree	Joints
13.5:1	614.4	X
48.0:1	2184.5	A
67.82:1	3086.56	B, C
150.0:1	6826.7	Y, Z

Caution: Three wrong conversion constants exist in this project's history

Do not use any of them. (1) "≈3300 counts per 90°" describes the *motor shaft*, not the joint — through 67.82:1 that same motion is only ≈ 1.07° of joint travel. (2) `COUNTS_PER_REV = 16050` — the encoder is 14-bit, so 16,384 (2^{14}) is correct and source-verified. (3) `pulses_per_degree = -813.0` in `mks_driver.py`, off by roughly 3.8×, with an accompanying "safety cage" that estimated limits using a different multiplier than the one actually transmitted — so it could not reliably block an unsafe command. Use only the formula above.

8 Safety

Action	Frame	Example (Joint A, 0x04)
Enable coils	F3 01 <crc>	cansend can0 004#F301F8
Disable coils	F3 00 <crc>	cansend can0 004#F300F7
Emergency stop	F7 <crc>	cansend can0 004#F7FB

Table 5: Recompute the checksum for the joint you are actually driving.

Caution: Compute your joint's e-stop frame before you need it

A previous version of this documentation gave Joint C's emergency stop as `cansend can0 001#F7F8` — addressed to CAN ID `0x01`. It would have done nothing to the actual motor. Derive the frame for the joint in front of you, write it down, and keep it in a second terminal before commanding motion.

Caution: Dropping coils on a gravity-loaded joint lets it fall

Several scripts in this package disable the motor in a `finally` block. On an unloaded bench motor that is harmless; on an assembled arm it removes holding torque. Know which you have before running one.

9 Troubleshooting

Symptom	Likely cause / fix
`can0` does not exist	`slcand` died, usually because the adapter re-enumerated. Re-run the bring-up in Section 4. Check `/dev/serial/by-id/` for the current device.
Adapter present but `can0` missing after replug	Expected — `slcand` does not restart itself. A udev rule keyed on the adapter's serial can automate this.
`ip` shows `bitrate 0`	Normal on slcan. Not a fault.
No response from any ID	Check the panel's CAN ID, CAN rate, and CANRSP; check `CAN_H`/`CAN_L` and termination; confirm the driver is powered. If still silent, see Section 9.1.
Command acknowledged, motor does not move	Distinguish "not being driven" from "driven but cannot move the load" — the bus cannot tell these apart. Disable coils and turn the joint by hand.
Motion for a few degrees, then nothing in either direction	If it cannot even *back off*, suspect either a bind stiff enough to absorb full torque, or a marginal motor phase connection. Reseating a connector once took a joint's travel budget from 1.05° to 3.85°.

Symptom	Likely cause / fix
Gritty resistance by hand, stalls under power	Gear mesh needs lubrication. Use a plastic-safe grease (PTFE or silicone). Petroleum jelly works as a diagnostic but migrates once warm.
Motor warm but not hot	Normal. Thermal shutdown makes a motor too hot to touch; "warm" is a stepper doing its job.
Driver stops responding mid-test	Check power and connectors first. This has been caused by an accidental unplug more than once.

Table 6: Failure modes actually observed on this arm.

9.1 Total silence with a powered driver: the RS485 variant

The MKS SERVO42D/57D ship in both CAN and RS485 variants. The RS485 boards have **no CAN transceiver at all** and will never appear on the bus at any bitrate or address. Joint X's driver turned out to be the RS485 variant, which is why it has never communicated. If a joint is silent across a full ID scan and a bitrate sweep, with power and wiring confirmed, **check the board's part number before spending more time in software**.

10 What This Guide Supersedes

`arctos_joint_b_guide.tex` was removed and its generally-applicable content consolidated here. The following items were **corrected** rather than copied, so do not re-derive them from any surviving copy of the old document:

- **Encoder opcode.** The old guide standardized on `0x30` and described `0x31` as unverified. This is backwards: `0x31` is what the real C++ driver uses and what every script in this package now uses, with the 48-bit layout given in Section 6.3.
- **Bitrate verification.** The old guide instructed the reader to run `ip link set can0 up type can bitrate 500000` and then verify by looking for `bitrate 500000`. Neither works on slcan; the interface correctly reports `bitrate 0`.
- **Bus error counters.** The old troubleshooting advice leaned on bus-error counts that the slcan driver never populates.
- **Motion command.** The old guide's safe-first-motion procedure was built on `0xF5`, whose absolute-vs-relative behaviour it correctly flagged as unresolved. That question is now moot: use `0xFD`.
- **Joint-specific framing.** Checksums, e-stop frames and panel settings are now presented per-joint rather than hard-coded to Joint B's `0x05`, which was a copy-paste hazard.